NEXUS

Causal Intelligence Platform — Full Architecture

Built on IBM Bob IDE · Claude · Neo4j · React

Nexus is the first software comprehension platform built on a formal computational model of organizational causality. While every other AI coding tool in 2026 competes on velocity (how fast to write the next line), Nexus competes on comprehension — making the invisible causal chains inside a codebase visible, queryable, and actionable.

Core insight: Software systems accumulate causally invisible debt. A constraint baked into a 2021 decision silently shapes the 2026 architecture. Nexus makes that chain visible.

1. System Architecture — The Five Layers

Nexus is composed of five processing layers. Layers 1–3 require Bob IDE's full-repository context and cannot be replaced by any file-level AI. Layers 4–5 use Claude for deep semantic reasoning and are run in parallel to save time.

Layer 1 — Decision Provenance Engine	◆ Bob IDE *primary*
• Ingests entire repo, all git history, all linked GitHub/Jira issues, all PR descriptions, all Confluence/Notion pages.	

Layer 2 — Causal Temporal Graph (CTG)	◆ Bob IDE *primary*
• DPRs become nodes in a Directed Acyclic Graph (DAG) with time as a structural dimension.	

Layer 3 — Assumption Decay Monitor	◆ Bob IDE *continuous monitoring*
• Every DPR contains classified assumptions: quantitative, environmental, dependency, and team assumptions.	

Layer 4 — Counterfactual Simulation Engine	◆ Claude *deep reasoning*
• Given the CTG, Bob/Claude reasons over alternative decision timelines.	

Layer 5 — Organizational Risk Forecast	◆ Claude *semantic analysis*
• Knowledge concentration analysis: 'Engineer X holds 73% of causal knowledge about auth — their departure is a critical SPOF.'	

2. Data Flow & Storage

Inputs consumed by Bob (Layer 1 ingestion)

Source	What Bob extracts	Format
Git history (all commits)	Decision timelines, constraint evolution, author attribution	git log / GitHub API
GitHub / Jira issues + PRs	Rejected alternatives, debate context, explicit constraints	REST/GraphQL API
PR descriptions & review comments	Intent, trade-off reasoning, reviewer pushback	GitHub API
Confluence / Notion docs	Architectural decisions, ADRs, runbooks	Confluence API
Full repository code	Implicit constraints, structural patterns, dependency graph	Bob full-repo context

Outputs produced

Output	Contents	Used by
nexus_data.json	Merged Bob + Claude output: DPRs, CTG edges, risk scores, remediation backlog	React dashboard
CTG database (Neo4j)	Full causal graph with time-indexed nodes and weighted edges	Live queries, Layer 4
Risk report PDF	6-month forecast: knowledge concentration, decay trajectories, remediation list	CFO / engineering leadership
React dashboard (Vercel)	Single-file no-backend UI: assumption drift alerts, CTG explorer, risk heatmap	Engineering team daily use
lablab.ai submission	Submission form text, Bob session export, demo script (word-for-word)	Hackathon judges

3. 24-Hour Implementation Timeline

Phase	Hours	Tool	Task
Phase 1	0 – 6h	Bob IDE	Paste Prompt 1 into Bob. Point at psf/requests repo. Let Bob run through all 4 ingestion steps fully — do not interrupt. This is the hardest compute Bob does. Output: structured DPR JSON.
Phase 2a	6 – 14h	Claude (parallel)	The moment Bob finishes Step 4, copy its full JSON output. Paste into Prompt 2 for Claude. Claude performs deep semantic reasoning: counterfactual tracing (Layer 4) and semantic risk analysis (Layer 5). Run this in parallel while Bob finishes.
Phase 2b	6 – 14h	Bob IDE	Bob continues to Step 5 and 6: risk score generation and remediation backlog. Both Bob and Claude finish during this window. Merge both JSON outputs into a single nexus_data.json file.
Phase 3	14 – 20h	Antigravity / Claude	Paste Prompt 3. Feed merged nexus_data.json. Entire React dashboard builds from this one file — no backend, no server. Push to Vercel in under 3 minutes.
Phase 4	20 – 24h	Bob IDE	Paste Prompt 4. Bob generates: lablab.ai submission form text, IBM Bob session export report (required for submission), and word-for-word demo script. Read demo script twice. Do not improvise.

Killer demo moment: psf/requests is a deliberate choice. The chardet to charset-normalizer migration in 2021 is a real historical incident where an encoding detection assumption was quietly changed after years of being foundational, causing real breakage. Bob will surface it as a Layer 3 Assumption Drift Alert. When it does, that is your demo's best beat — trace it live through the CTG.

4. What Already Exists — Use It, Don't Build It

The table below lists every component Nexus needs. The green rows are fully available off-the-shelf — install and configure only. The amber rows need light integration work (glue code, prompt engineering, or minor configuration). Only the items in Section 5 require building from scratch.

Component	Existing tool / library	Status	Effort
Full-repo ingestion	IBM Bob IDE (built-in)	**Ready**	0h
Git history parsing	GitPython / GitHub API	**Ready**	1h setup
Jira/GitHub issue fetch	PyGitHub + Jira REST API	**Ready**	2h setup
Confluence/Notion fetch	Confluence REST API / Notion SDK	**Ready**	2h setup
Graph database (CTG)	Neo4j Community (free) + neo4j-driver	**Ready**	2h setup
Vector search (semantic)	Weaviate Cloud (free tier)	**Ready**	1h setup
LLM reasoning (L4, L5)	Claude Sonnet via API	**Ready**	0h
Bob IDE prompting (L1-L3)	IBM Bob IDE — Prompt 1	**Ready**	0h
React dashboard scaffold	Vite + React + Tailwind	**Ready**	1h
Dashboard deploy	Vercel CLI (free)	**Ready**	3 min
PR/commit diff parsing	PyDriller (pip install)	**Ready**	1h
JSON schema validation	Pydantic v2	**Ready**	1h
Risk report PDF	ReportLab (pip install)	**Ready**	2h
Demo repo target	psf/requests (public GitHub)	**Ready**	0h
ADR parsing	adr-tools / custom regex	**Configure**	1h
Assumption classifier	Claude prompt (few-shot)	**Configure**	2h
Blast radius scoring	NetworkX (pip install)	**Configure**	3h
Knowledge concentration	git-fame (pip install)	**Configure**	2h

5. What You Must Build — Easiest Possible Way

These are the components that have no off-the-shelf equivalent. Each entry includes the absolute easiest implementation path — no over-engineering.

What to build	Easiest approach	Time est.
DPR extractor (Decision Provenance Records)	Bob Prompt 1 does this entirely. Bob reads full repo + git + docs and outputs structured JSON DPRs. Your only job: write a Pydantic schema (class DPR) to validate and store the output. ~50 lines of Python.	2h
CTG builder (Causal Temporal Graph)	Take Bob's DPR JSON. For each DPR, create a Neo4j node. For each causal relationship Bob identifies between DPRs, create a directed edge. Use the official neo4j-driver Python package. The entire ingestion script is ~80 lines. No graph ML needed — just cypher CREATE statements.	3h
Assumption Decay Monitor	Two parts: (1) A Python script that runs nightly, re-fetches the live repo state via GitHub API, and re-sends each DPR assumption to Claude with the prompt: 'Does this assumption still hold given this current state? Return JSON: {holds: bool, confidence: float, reason: str}'. (2) A NetworkX traversal that computes blast radius for each flagged assumption (count downstream nodes). ~100 lines total.	4h
nexus_data.json merger	A 30-line Python script: load Bob's output JSON, load Claude's output JSON, merge into one dict with keys: dprs[], ctg_edges[], assumptions[], risk_scores[], remediation_backlog[]. Write to nexus_data.json. This is the only file the React dashboard ever reads.	1h
React dashboard	Prompt 3 into Claude/Antigravity with nexus_data.json attached. Claude writes the entire single-file React component. You review, paste into src/App.jsx in a Vite project, run 'vercel --prod'. No backend. No API. The JSON file IS the backend.	2h
Counterfactual simulation (Layer 4)	Prompt 2 for Claude. You pass Bob's full DPR JSON + CTG edge list. Claude's prompt: 'Given this causal graph, simulate: if [decision X] had been made differently at [date], trace which downstream decisions would have changed and describe the resulting architecture.' Claude returns structured markdown. Parse with regex into JSON. ~20 lines of parsing code.	3h (Claude does the reasoning)
Knowledge concentration report	Run git-blame across every file in the repo. Map each line to an author. For each DPR node in the CTG, compute which author(s) have the most blame lines in the files that DPR touches. Normalize to percentage. Flag anyone above 60% on a critical-path DPR. ~60 lines of Python using git-fame output.	2h

What to build	Easiest approach	Time est.
Submission export (lablab.ai)	Prompt 4 into Bob. Bob generates: (1) submission form text, (2) Bob session export in required format, (3) word-for-word demo script. Your only job is copy-paste. Read the demo script twice. The psf/requests chardet story IS your demo — do not replace it with anything else.	1h

6. Full Tech Stack Reference

Category	Technology	Install / link
Primary AI (L1–L3)	IBM Bob IDE	lablab.ai hackathon access
Secondary AI (L4–L5)	Claude Sonnet 4 (claude-sonnet-4-20250514)	api.anthropic.com
Graph database	Neo4j Community 5.x	pip install neo4j
Vector search	Weaviate Cloud free tier	weaviate.io/developers/weaviate
Repo parsing	PyDriller + GitPython	pip install pydriller gitpython
Commit author stats	git-fame	pip install git-fame
Graph algorithms	NetworkX	pip install networkx
Data validation	Pydantic v2	pip install pydantic
GitHub / Jira API	PyGitHub + atlassian-python-api	pip install PyGitHub atlassian-python-api
Frontend	React + Vite + Tailwind CSS	npm create vite@latest
Deploy	Vercel CLI	npm i -g vercel
PDF output	ReportLab	pip install reportlab
Target demo repo	psf/requests (public)	github.com/psf/requests

7. Research & Business Framing

Theoretical foundations (for ICLR / IEEE submission)

Distributed Cognition (Hutchins, 1995): Complex cognitive tasks are performed by systems of humans + artifacts + environments. A software organization is a distributed cognitive system. The codebase, git history, docs, and issue trackers are its external cognitive artifacts. Nexus is the first computational system designed around this model.

Causal Inference (Pearl, 2009): The do-calculus distinguishes correlation from causation. Nexus applies causal reasoning — not statistical correlation — to software organizational data. This is the novel contribution over existing MSR (Mining Software Repositories) work, which is purely correlational.

Invariant Inference (Daikon, Ernst et al., 2007): Traditional invariant detection is dynamic and syntactic. Nexus performs semantic invariant detection on architectural decisions — detecting when the invariants assumed at decision time no longer hold in the live system. LLM as semantic invariant detector over natural language artifacts is a novel formulation.

Competitive business frame

Every AI coding tool in 2026 competes on velocity — how fast from intent to implementation. IBM Bob + Nexus competes on a fundamentally different axis: comprehension — how deeply does an AI system understand why a software organization is the way it is?

Velocity tools produce more code faster. Comprehension tools prevent the catastrophic failures that velocity tools ironically accelerate — because the faster you build on top of an undocumented, causally opaque foundation, the faster you accumulate the assumptions that will eventually detonate.

Market	Size	Competition
Velocity tooling (Copilot, Cursor, Devin)	$6B developer tools market	Extremely crowded
Comprehension tooling (Nexus)	$300M-per-outage enterprise reliability market	Completely uncontested

Why Bob is structurally irreplaceable

Causal tracing across a codebase requires understanding semantic relationships between components that don't share keywords, don't call each other directly, and may exist in completely different files written years apart. A decision about database indexing strategy in 2020 causally constrains the API pagination design in 2023, which causes mobile app performance degradation in 2025.

No static analysis tool can trace this. No file-level AI can see it. Only a system that reads the entire repository simultaneously — understanding the full semantic architecture — can detect this connection. Full repository context is not a feature of Bob. It IS Bob. Nexus is the application that finally deserves it.